

# Lesson 2 — What Each Notebook Shows

Notebook guide, lesson 2 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

9 October 2026 · three notebooks

## Contents

|                                                                        |          |
|------------------------------------------------------------------------|----------|
| <b>Notebook 01 — 01_exploratory_data_analysis</b>                      | <b>1</b> |
| Looking at a dataset before touching it . . . . .                      | 1        |
| <b>Notebook 02 — 02_scaling_encoding_and_pipelines</b>                 | <b>2</b> |
| Scaling, encoding, and the pipeline that holds them together . . . . . | 2        |
| <b>Notebook 03 — 03_leakage_in_preprocessing</b>                       | <b>3</b> |
| Two leaks that look nothing like leaking . . . . .                     | 3        |

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

---

## Notebook 01 — 01\_exploratory\_data\_analysis

### Looking at a dataset before touching it

**What it is for.** To build the picture that every later decision depends on: how big the data is, what each column means, how much is missing, where the outliers live, and which variables move together. Nothing is fitted here — the whole dataset is still unsplit, on purpose.

**The data.** A synthetic telecom dataset generated by `churn_data.py`: 2,000 customers, 8 columns, and the question of whether each one churned. Synthetic **so that the ground truth is known** — which is what makes notebook 3's demonstration conclusive.

### The concepts, in order.

1. **.info() and .describe() first.** They surface the problems in seconds: `tenure_months` runs from **−3 to 999**, `monthly_charges` to **3344.74** with a standard deviation of 171 against a median of 63.
2. **The baseline.** Churn rate **0.194**, so always predicting “no churn” is right **80.6%** of the time.

3. **Missingness, with its mechanism.** `age` 160 rows (**8.0%**), missing completely at random (MCAR); `num_support_calls` 96 rows (**4.8%**), missing at random (MAR) — missing more often for long-tenure customers.
4. **What filling a gap would cost**, measured against the derivation:

| fraction filled | correlation | kept  | $\sqrt{1-p}$ |
|-----------------|-------------|-------|--------------|
| 0.0%            | 0.700       | 1.000 | 1.000        |
| 8.0%            | 0.671       | 0.959 | 0.959        |
| 40.0%           | 0.543       | 0.776 | 0.775        |

The last two columns agree to a thousandth at every fraction: **the attenuation formula arriving on data.**

5. **Outliers from scratch**, both rules implemented from their formulas. On `monthly_charges`: z-score flags 20, Tukey flags 32, and they agree on 20.
6. **The twist that makes the section.** The contamination argument is *right about the damage and wrong about the consequence*: the errors inflate `s` from **17.23 to 171.25** and push the fence from **115 to 593** — but the smallest genuine billing error is **780.1**, so the ruined fence still catches all 20. Tukey catches the same 20 plus **12 ordinary customers** (values like 15.0 and 128.4).
7. **What neither rule can see.** Six negative tenures, none flagged by the interquartile range (IQR) rule, because `-3` sits comfortably inside a column spanning 0 to 72. The domain rule `0 ≤ tenure ≤ 120` flags **10 rows**.
8. **Correlations and cardinality.** `tenure_months` and `num_support_calls` correlate with churn, `age` barely does — matching how the data was generated. And `zip_code` has **493 levels over 2,000 rows**, which is the fuse for notebook 3.

**The two-minute version.** *This notebook is the look before the touch. Two calls, `.info()` and `.describe()`, already show a tenure of 999 and one of `-3`, and a monthly charge of 3,344. Then the missingness: 8% of `age`, missing completely at random, and 4.8% of support calls, missing because of tenure. The table of imputation costs matches the square-root-of-one-minus-p formula to a thousandth. Then the two outlier rules disagree — 20 against 32 — and the interesting part is that the z-score rule was right by luck: the errors inflated its ruler fivefold, and it catches them only because they are enormous. Finally, six negative tenures that no statistical rule flags at all, because being impossible is not the same as being unusual.*

---

## Notebook 02 — 02\_scaling\_encoding\_and\_pipelines

### Scaling, encoding, and the pipeline that holds them together

**What it is for.** To build the preprocessing that notebook 1 argued for, and to prove the scaling half rather than assert it. It implements handout sections 5, 6 and 8.

**The concepts, in order.**

1. **Split first.** 1,500 training rows against 500; baseline 0.806. `zip_code` is dropped for now — it returns, deliberately, in notebook 3.
2. **Why scale, measured.** Feature standard deviations 0.98 and 10.23, a **variance ratio of 110**. Raw features tolerate a learning rate of 0.1 and reach a loss of 0.4183; scaled features tolerate **2.0**, twenty times larger, and reach 0.4155 in a fraction of the iterations.
3. **What happens at the wrong rate.** Raw features at  $\eta = 2.0$ : the loss starts at 0.6931 and then **swings between 0.69 and 8.29** — still finite after 200 steps and no closer to settling. No exception, no warning, no convergence.
4. **StandardScaler against MinMaxScaler**, both fitted on training data only. MinMax is the more fragile: one extreme billing error stretches the whole  $[0, 1]$  range and compresses every ordinary customer into a sliver of it.
5. **The dummy variable trap.** All  $k$  dummies plus an intercept: **4 columns, rank 3**. With  $k - 1$  dummies: 3 columns, rank 3. The redundancy the derivation predicts, on the actual matrix.
6. **Why zip\_code waits.** 493 levels over 2,000 rows, **4.1 rows per level**, and **99.80% of the one-hot matrix is zero**.
7. **ColumnTransformer + Pipeline.** Numeric columns imputed then scaled, categoricals imputed then one-hot, all in one object: accuracy **0.820** against a baseline of 0.806, area under the receiver operating characteristic curve (AUC) **0.751** (the chance a random churner is scored above a random non-churner; 0.5 is a coin flip). The small accuracy gain is the imbalance doing what lesson 1 warned it would.
8. **Feature engineering, inside the pipeline.** `total_paid` moves AUC from **0.7514** to **0.7548**, a gain of 0.0034 — and to **0.7439** when the domain rule is applied first, a loss of 0.0075.

**The two-minute version.** *This is where the preprocessing gets built and one claim gets proved. The two features differ in variance by a factor of 110, and that is exactly what decides the learning rate: raw data tolerates 0.1, scaled data tolerates 2.0 and converges far sooner. Push the raw data to 2.0 and the loss oscillates between 0.69 and 8.29 forever — no error, just never converging. Then encoding: all the dummies plus an intercept give four columns of rank three, which is the dummy trap on the real matrix. Everything goes into one ColumnTransformer and Pipeline, AUC 0.751. The last section adds an engineered feature and gains 0.003 — and loses 0.008 when the data is cleaned first, because the feature had been multiplying the contamination.*

---

## Notebook 03 — 03\_leakage\_in\_preprocessing

### Two leaks that look nothing like leaking

**What it is for.** Lesson 1 broke train/test independence in the most visible way. This notebook breaks it in the two ways that look like ordinary preprocessing.

#### The concepts, in order.

1. **Leak 1 — imputing with the whole dataset.** 128 training rows have a missing `age`, and **94 of the 128 had at least one test row among their five donors**. The notebook even names one: training row 1922 was imputed partly from test row 1127.
2. **And then the honest surprise.** Leaky AUC **0.7553**, honest AUC **0.7546** — a gap of seven ten-thousandths. Rerun over 20 splits and the gap **changes sign in 14 of them**,

mean  $-0.0003$ . **The gap is noise.** The contamination is real and measurable in the donors; its effect on this score is not measurable at all, which is a more honest and more useful result than a dramatic number would be.

3. **Leak 2 — encoding with the whole dataset.** `zip_code` carries no signal by construction. Encoded before the split: accuracy 0.846, **AUC 0.891**. Encoded honestly with a cross-fitted `TargetEncoder` inside the pipeline: accuracy 0.820, **AUC 0.751** — which is what the model scored in notebook 2 without the column at all, exactly as it should be.
4. **Why small groups are what make it dangerous.** The notebook finds a code with one customer: their churn label is 0, and their leaky encoded feature is 0.000. **For that row the feature is not a predictor of the label. It is the label.**

**The two-minute version.** *Two leaks that look like preprocessing. First, imputing `age` with a  $k$ -nearest-neighbours imputer fitted before the split: 94 of the 128 affected training rows borrowed a value from a test row. And the AUC barely moves — seven ten-thousandths, which flips sign across splits, so the leak is real and its effect here is unmeasurable. That is worth saying plainly: the damage is in the method, not in the number. Second, target-encoding `zip_code` before the split takes AUC from 0.751 to 0.891 — fourteen points out of a column that contains nothing, by construction. The mechanism is at the end: a zip code with one customer gets an encoded value identical to that customer's label. The feature is the answer.*